Warm up

The following programs compute the same arithmetic. Without compiler optimisation, which do you think is faster, and why?

```
int foo1(int a, int b) {
  return a + b;
}
```

```
int foo2(int a, int b) {
  int x = a + b;
  return x;
}
```

Warm up

The following functions compute the same arithmetic. Without compiler optimisation, which do you think is faster, and why?

```
int foo1(int a, int b) {
  return a + b;
}
```

```
int foo2(int a, int b) {
  int x = a + b;
  return x;
}
```

- Program A is faster because it does not need to allocate space for the local variable `x` and store the intermediate result

```
foo1:
  str   w0, [sp, 12]
  str   w1, [sp, 8]
  ldr   w1, [sp, 12]
  ldr   w0, [sp, 8]
  add   w0, w1, w0
```

```
foo2:
  str   w0, [sp, 12]
  str   w1, [sp, 8]
  ldr   w1, [sp, 12]
  ldr   w0, [sp, 8]
  add   w0, w1, w0
  str   w0, [sp, 28]
  ldr   w0, [sp, 28]
```

Precept 16: Introduction to the ARMv8 assembly language

Polly Ren

COS 217: Introduction to Programming Systems
Spring 2026
Princeton University

March 31, 2026

Overview

Assembly languages

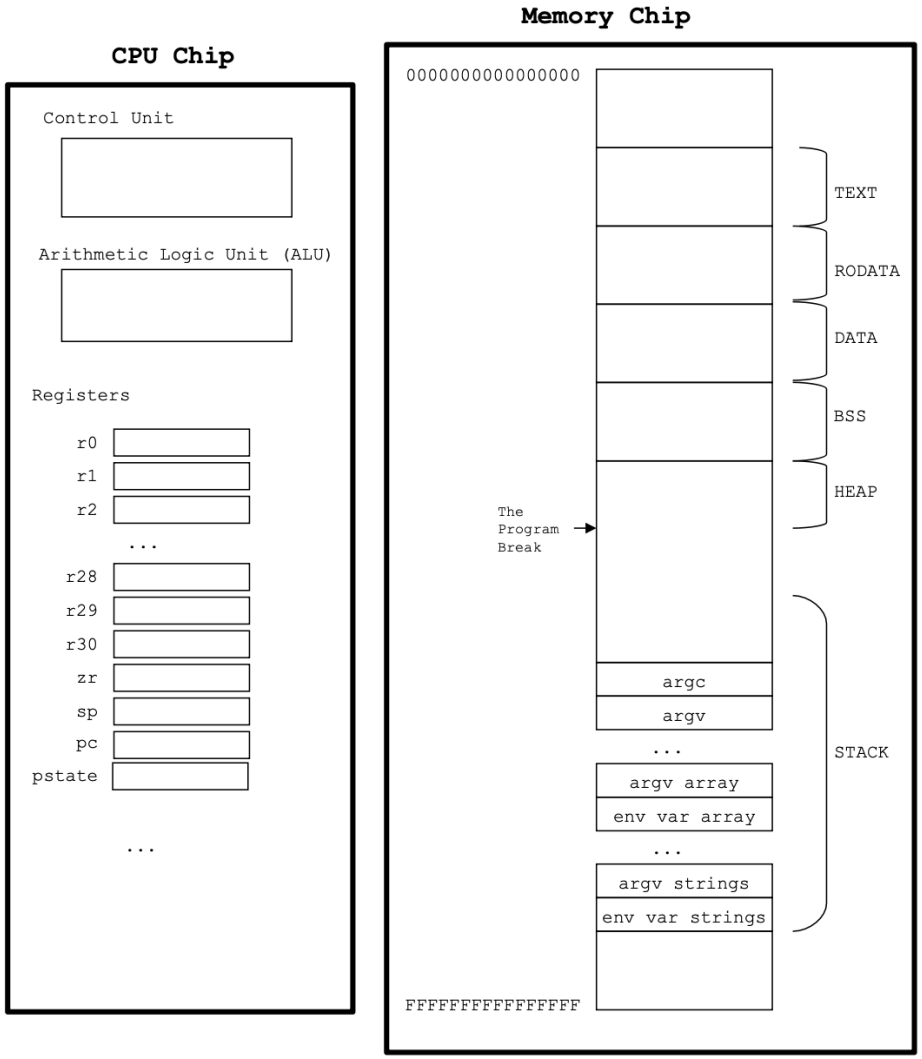
- An *assembly language* is a low-level programming language that provides a human-readable representation of a CPU's *instruction set architecture (ISA)*
  - One assembly instruction → roughly one machine instruction
  - Contrast with C: one C statement compiles to many instructions
  - Assembler translates assembly source (`.s`) to machine code (`.o`)
- Two broad ISA philosophies:
  - *CISC* (Complex Instruction Set Computer): many specialised instructions, instructions can operate directly on memory (e.g., x86)
  - *RISC* (Reduced Instruction Set Computer): simpler uniform instruction set, load / store architecture (i.e., computation happens in registers, memory accessed only via explicit loads and stores)
- We will work with the ARMv8-AArch64 ISA
  - RISC architecture
  - Used in mobile devices (e.g., iPhones, Android phones) and in laptops and servers (e.g., Apple M1/M2, AWS Graviton)

Why bother with assembly language?

- To understand and compose system-level code (e.g., preprocessor, compiler, assembler, linker, operating systems, etc.)
- To compose faster application-level code (e.g., `adcs` instruction to add large integers)
- To develop a better understanding of C, and to appreciate the language's design
- Most importantly: to be aware of what is going on "under the hood", even below the level of C

Overview

An oversimplified view of the computer



- Memory: stores data
  - Think of as a large array, indexed by addresses (64-bit in ARMv8-AArch64) $\rightarrow 2^{64}$ bytes
  - Divided into segments (e.g., text, rodata, data, bss, heap, stack)
- CPU: executes instructions
  - Control unit: fetches and decodes instructions
  - ALU: performs arithmetic and logical operations
  - Registers: small, fast storage locations for data and addresses

Memory vs. registers

- When the program is running, data is stored in memory or in registers
- Registers are very fast
  - Because they are located on the CPU chip, where the control unit and ALU are also located
- Memory access is much slower (sometimes more than 100×)
  - Control unit and ALU are on separate chip from memory
  - Need to transfer data between CPU and memory via a bus, which is much slower than accessing registers
- Why are closer storage locations faster? Speed of light!
- There are also other levels of storage (e.g., cache, disk)
- The general pattern in assembly code:
  - Load data from memory into registers
  - Perform operations on data in registers
  - Store results back to memory (if needed)

General-purpose registers

- ARMv8 has 31 general-purpose registers (r0-r30)
- In code, registers are referred to as `xN` or `wN` (e.g., `x0`, `w0` for `r0`)
  - `x0`–`x30` refers to the full 64-bit registers
  - `w0`–`w30` refers to the lower 32 bits of the registers
- There are conventions for how registers are used in procedure calls:
  - `r0`: used to return a value from a function
  - `r0`–`r7`: used for passing arguments to functions, and for temporary values within functions
  - `r8`: indirect result location parameter (we will not use)
  - `r9`–`r15`: used for temporary values within functions
  - `r16`–`r17`: intra-call scratch registers (we will not use)
  - `r18`: platform register (we will not use)
  - `r19`–`r28`: used for optimised program design
  - `r29`: frame pointer (we will not use)
  - `r30`: link register, used to store return address

Special-purpose registers

- Four special-purpose registers:
  - `sp`: stack pointer, points to the top of the stack
    - Stack grows downwards (towards lower addresses)
    - Pushing onto stack decrements `sp`, popping from stack increments `sp`
  - `pc`: program counter, points to the instruction being executed
  - `zr`: zero register, always reads as 0 and discards writes
  - `pstate`: processor state register, holds flags and control bits
    - Flags: N (negative), Z (zero), C (carry), V (overflow)
    - Flags are set by certain instructions and can be used for conditional execution

Overview

Parts of an assembly program

- Comments: `// this is a comment`
  - Can be used to explain the code in plain English, or can annotate with corresponding C code
- Labels: `label:`
  - Used to mark a location in memory with a name, which can be used for jumps and branches
  - Importantly: labels are not instructions, only markers for the assembler and linker to resolve addresses
  - Typically used for data, text, rodata, bss segments' data
- Directives: `.section .text`, or `.string "hello"`, etc.
  - Used to specify how the assembler should process the code, e.g., which segment to place the following instructions or data in, alignment, etc.
  - Does not correspond to actual machine instructions
- Instructions: `add x0, x1, x2`, or `ldr x0, [x1]`, etc.
  - Correspond to actual machine instructions that the CPU executes
  - Typically in the text segment

Types of instructions

- Load instructions: copy data from memory to registers (e.g., `ldr`)
  - `ldr w0, [x1]`: load 32 bits from memory address in `x1` into `w0`
  - `ldr x0, [x1]`: load 64 bits from memory address in `x1` into `x0`
  - Data flows from right to left
- Arithmetic and logical instructions: perform operations on data in registers (e.g., `add`, `sub`, `and`, `orr`, etc.)
  - `add w0, w0, 1`: `w0` = `w0` + 1
  - `add w2, w0, w1`: `w2` = `w0` + `w1`
  - Data flows from right to left
- Store instructions: copy data from registers to memory (e.g., `str`)
  - `str w0, [x1]`: store 32 bits from `w0` into memory address in `x1`
  - `str x0, [x1]`: store 64 bits from `x0` into memory address in `x1`
  - Data flows from left to right(!)
- (Also: control flow instructions: jump, branch, call, return, etc.)

Your turn: writing assembly

- Copy from one register to another: `w1 = w2`, `x4 = x3`
- Load: load four bytes into `w1` from address in `x2`
  - What is in `w1` after this instruction is executed?
  - What is in `x2` after this instruction is executed?
  - What if we loaded into `w1` from the address in `x1` instead?
- More load: load eight bytes into `x1` from address in `x2`
  - What is in `x1` after this instruction is executed?
  - What is in `x2` after this instruction is executed?
  - What if we loaded into `x1` from the address in `x1` instead?
- Arithmetic: `x3 = x1 + x2`, `x2 = x2 + 1`

Your turn: writing assembly (possible solutions)

- Copy from one register to another: `w1 = w2`, `x4 = x3`
  - `mov w1, w2`
  - `mov x4, x3`
  - Note: `mov` is a pseudo-instruction that the assembler translates into an actual instruction (e.g., `orr w1, w2, wzr` for 32-bit registers, `add x4, x3, xzr` for 64-bit registers)
- Load: load four bytes into `w1` from address in `x2`
  - `ldr w1, [x2]`
  - What is in `w1` after this instruction is executed? The 4 bytes stored in memory at the address held in `x2`
  - What is in `x2` after this instruction is executed? `x2` unchanged
  - What if we loaded into `w1` from the address in `x1` instead? `ldr w1, [x1]`
    - `w1` is the low 32 bits of `x1`, so the load overwrites the pointer in `x1`
    - `x1` now holds the loaded value, not the original address

Your turn: writing assembly (possible solutions, cont.)

- More load: load eight bytes into `x1` from address in `x2`
  - `ldr x1, [x2]`
  - What is in `x1` after this instruction is executed? The 8 bytes stored in memory at the address held in `x2`
  - What is in `x2` after this instruction is executed? `x2` unchanged
  - What if we loaded into `x1` from the address in `x1` instead? `ldr x1, [x1]`: as before, the load overwrites the pointer in `x1`, so `x1` now holds the loaded value, not the original address
- Arithmetic: `x3 = x1 + x2`, `x2 = x2 + 1`
  - `add x3, x1, x2`
  - `add x2, x2, 1`
    - Or: `add x2, x2, #1` (# indicates immediate value, but not required)

`ldr` vs. `ldrb`, `str` vs. `strb`

- `ldr` and `str` load and store 32 or 64 bits (4 or 8 bytes)
- `ldrb` and `strb` load and store 8 bits (1 byte)
- What happens if we use `ldrb` to load from an address that contains more than 1 byte of data?
  - Only least significant byte is loaded into register
- What happens if we use `strb` to store into an address that already contains data?
  - Only least significant byte of the register is stored into memory, the rest of the data at that address remains unchanged
- What instructions would you use for the following:
  - Load 1 byte from memory addressed by `x2`, then zero-extend it to `w1`
    ```
    ldrb w1, [x2]
    ```
  - Store 1 byte from `w1` into memory addressed by `x2`
    ```
    strb w1, [x2]
    ```

Developing an assembly program

- Create a `.s` file for the assembly source code: `emacs pgm.s`
- Assemble and link the program: `gcc217 pgm.s -o pgm`
  - Directly invokes assembler and linker, skips preprocessor and compiler
  - `gcc` recognises the `.s` extension and knows to invoke the assembler
- Run the program: `./pgm`

- Note: `gcc217 -S pgm.c` generates the assembly code for `pgm.c` and saves it in `pgm.s`
  - Can be used to see how the compiler translates C into assembly
  - Compiler optimisations may make the generated assembly code more complex and less readable
  - Silently overwrites (clobbers) `pgm.s` if it already exists
  - Caveat: do not use this to generate assembly for Assignment 5

`hello.s`: assembly time

- The assembler's job is to translate `hello.s` into `hello.o` (object file)
- Object files have sections, denoted by the `.section` directive in the assembly code
  - `.section .text`: contains executable instructions
  - `.section .rodata`: contains read-only data (e.g., string literals)
  - `.section .data`: contains initialized data (e.g., global variables)
  - `.section .bss`: contains uninitialised data
- Heap and stack are managed at runtime, so do not have corresponding sections in the object file
- (The BSS is technically managed by the runtime loader, so the zeroed data is not physically stored in the object file, but the BSS section is still used to reserve space for it)
- See `hello.s` and `hello.c`

`hello.s`: interesting bits

- `greetingStr:`: defines a label for the string literal
- `.string "hello, world\n"`: places a 14-byte string (including NUL) in rodata
- `.equ MAIN_STACK_BYTECOUNT, 16`: defines a constant for the size of the main stack frame
- `.global main`: makes the `main` label visible to linker
- `.size main, (. - main)`: defines the size of the `main` function as the distance from here (`.`) to the `main` label
  - Not generally necessary for execution, but needed for `gdb` and other debugging tools to know where the function ends=

`hello.s`: link time

- The linker creates an executable from the object files
- Resolution: resolves symbol references to their addresses
  - Find code referenced by global symbols, and merge into program image
  - Places definition of `_start()` into executable to serve as entry point
  - In this case:
    - Find `printf.o` (in `libc.a`) and merge its sections into `hello.o`
- Relocation: updates the code and data to reflect the resolved addresses
  - Replace reference to section offsets with real (virtual) memory addresses
  - Replace references to externally-defined functions with real (virtual) memory addresses
  - In this case:
    - In `bl`, replace reference to `printf` with a memory address
    - In `adr`, replace reference to `greetingStr` with a memory address
    - Both are displacements from the program counter

hello.s: run time

- The executable is loaded into memory and executed by the OS
- Execution starts at the entry point defined by the linker (`_start()`)
  - `_start()` sets up the stack and stack pointer, and calls `main()`
- Function prolog: will discuss later

```
sub sp, sp, MAIN_STACK_BYTECOUNT
str x30, [sp]
```

- `adr x0, greetingStr`: load address of `greetingStr` into `x0`
- `bl printf`: branch and link to `printf`
  - `printf` uses contents of `x0` as argument (the string to print)
- `mov w0, 0`: set return value of `main()` to immediate value 0
- Function epilog: reverse of prolog, will also discuss later

```
ldr x30, [sp]
add sp, sp, MAIN_STACK_BYTECOUNT
```

- `ret` returns from `main()` to `_start()`, which cleans up and exits

Overview

Summary

- What is assembly language and why we care about it
- The architecture of the computer: memory, CPU, registers
- The ARMv8 assembly language: instructions, directives, labels, comments
- Introduction to writing assembly code and the assembly process
- Dates to remember:
  - Recommended: find an Assignment 4 partner asap
  - Assignment 4: due Tue, 4/7 at 9pm
  - Assignment 4 Quiz: Mon, 4/13 in lecture